

AI Assisted Coding

LAB ASSIGNMENT – 5.1 and 6.1

Name: N.Trinandini
Batch: 32
Hall Ticket No: 2303A52071

Task 1:

Employee Data: Create Python code that defines a class named `Employee` with the following attributes: `empid`, `empname`, `designation`, `basic\_salary`, and `exp`. Implement a method `display\_details()` to print all employee details. Implement another method `calculate\_allowance()` to determine additional allowance based on experience:

- If `exp > 10 years` → allowance = 20% of `basic\_salary`
- If $5 \leq \text{exp} \leq 10$ years → allowance = 10% of `basic\_salary`
- If `exp < 5 years` → allowance = 5% of `basic\_salary`

Finally, create at least one instance of the `Employee` class, call the `display\_details()` method, and print the calculated allowance.

Code:

```
5.1and6.1.py > ...
1 class Employee:
2     def __init__(self, empid, empname, designation, basic_salary, exp):
3         self.designation = designation
4         self.basic_salary = basic_salary
5         self.exp = exp
6
7     def display(self):
8
9         print("Employee ID:", self.empid)
10        print("Employee Name:", self.empname)
11        print("Designation:", self.designation)
12        print("Basic Salary:", self.basic_salary)
13        print("Experience:", self.exp)
14
15    def calculate_allowance(self):
16        if self.exp > 10:
17            allowance = (20 / 100) * self.basic_salary
18        elif 5 < self.exp <= 10:
19            allowance = (10 / 100) * self.basic_salary
20        else:
21            allowance = (5 / 100) * self.basic_salary
22
23        total_salary = self.basic_salary + allowance
24        print("Allowance is:", allowance)
25        print("Total Salary is:", total_salary)
26
27
28
29 empobj1 = Employee(101, "John", "Manager", 50000, 12)
30 empobj1.display()
31 empobj1.calculate_allowance()
32
33 empobj2 = Employee(102, "Doe", "Developer", 40000, 7)
34 empobj2.display()
35 empobj2.calculate_allowance()
```

OUTPUT:

```
Total Salary is: 44000.0
PS D:\college\3-2\AIAC\AIAC LAB\04-02-2026> & "C:/Program Files/Python31
Employee ID: 101
Employee Name: John
Designation: Manager
Basic Salary: 50000
Experience: 12
Allowance is: 10000.0
Total Salary is: 60000.0
Employee ID: 102
Employee Name: Doe
Designation: Developer
Basic Salary: 40000
Experience: 7
Allowance is: 4000.0
Total Salary is: 44000.0
```

Task 2:

Electricity Bill Calculation- Create Python code that defines a class named `ElectricityBill` with attributes: `customer\_id`, `name`, and `units\_consumed`. Implement a method `display\_details()` to print customer details, and a method `calculate\_bill()` where:

- Units $\leq$ 100 $\rightarrow$ ₹5 per unit
- 101 to 300 units $\rightarrow$ ₹7 per unit
- More than 300 units $\rightarrow$ ₹10 per unit

Create a bill object, display details, and print the total bill amount.

CODE:

```

36
37 class ElectricityBill:
38     def __init__(self, customer_id, customer_name, units_consumed):
39         self.customer_id = customer_id
40         self.customer_name = customer_name
41         self.units_consumed = units_consumed
42     def display_details(self):
43         print("Customer ID:", self.customer_id)
44         print("Customer Name:", self.customer_name)
45         print("Units Consumed:", self.units_consumed)
46
47     def calculate_bill(self):
48         if self.units_consumed <= 100:
49             bill_amount = self.units_consumed * 5
50         elif 101 <= self.units_consumed <= 300:
51             bill_amount = (100 * 5) + (self.units_consumed - 100) * 7
52         else:
53             bill_amount = (100 * 5) + (100 * 7) + (self.units_consumed - 200) * 10
54
55         print("Total Bill Amount is:", bill_amount)
56
57 bill1 = ElectricityBill(1, "Alice", 250)
58 bill1.display_details()
59 bill1.calculate_bill()
60 bill2 = ElectricityBill(2, "Bob", 350)
61 bill2.display_details()
62 bill2.calculate_bill()
63

```

OUTPUT:

```

PS D:\college\3-2\AIAC\AIAC LAB\04-02-2026> & "C:/Program Files/Python314/python.exe" "d:/co
Customer ID: 1
Customer Name: Alice
Units Consumed: 250
Total Bill Amount is: 1550
Customer ID: 2
Customer Name: Bob
Units Consumed: 350
Total Bill Amount is: 2700
PS D:\college\3-2\AIAC\AIAC LAB\04-02-2026> 

```

Task 3:

Product Discount Calculation- Create Python code that defines a class named `Product` with attributes: `product\_id`, `product\_name`, `price`, and `category`. Implement a method `display\_details()` to print product details. Implement another method `calculate\_discount()` where:

- Electronics → 10% discount

- Clothing → 15% discount

- Grocery → 5% discount

Create at least one product object, display details, and print the final price after discount.

CODE:

```
class product:
    def __init__(self, product_id, product_name, price, category):
        self.product_id = product_id
        self.product_name = product_name
        self.price = price
        self.category = category

    def display_details(self):
        print("Product ID:", self.product_id)
        print("Product Name:", self.product_name)
        print("Price:", self.price)
        print("Category:", self.category)

    def calaculate_discount(self):
        if self.category.lower() == "electronics":
            discount = (10 / 100) * self.price
        elif self.category.lower() == "clothing":
            discount = (15 / 100) * self.price
        else:
            discount = (5 / 100) * self.price

        final_price = self.price - discount
        print("Discount is:", discount)
        print("Final Price is:", final_price)

prod1 = product(201, "Laptop", 60000, "Electronics")
prod1.display_details()
prod1.calaculate_discount()

prod2 = product(202, "Jeans", 2000, "Clothing")
prod2.display_details()
prod2.calaculate_discount()
```

OUTPUT:

```
Final Price is: 1700.0
PS D:\college\3-2\AIAC\AIAC LAB\04-02-2026> & "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/04-02-2026/5.1and6.1.py"
Product ID: 201
Product Name: Laptop
Price: 60000
Category: Electronics
Discount is: 6000.0
Final Price is: 54000.0
Product ID: 202
Product Name: Jeans
Price: 2000
Category: Clothing
Discount is: 300.0
Final Price is: 1700.0
PS D:\college\3-2\AIAC\AIAC LAB\04-02-2026> █
```

Task 4:

Book Late Fee Calculation- Create Python code that defines a class named `LibraryBook` with attributes: `book\_id`, `title`, `author`, `borrower`, and `days\_late`. Implement a method `display\_details()` to print book details, and a method `calculate\_late\_fee()` where:

- Days late $\leq 5 \rightarrow ₹5$ per day
- 6 to 10 days late $\rightarrow ₹7$ per day
- More than 10 days late $\rightarrow ₹10$ per day

Create a book object, display details, and print the late fees

CODE:

```
class LibraryBook:
    def __init__(self,book_id, title, author, borrower, days_late):
        self.book_id = book_id
        self.title = title
        self.author = author
        self.borrower = borrower
        self.days_late = days_late
    def display_details(self):
        print("Book ID:", self.book_id)
        print("Title:", self.title)
        print("Author:", self.author)
        print("Borrower:", self.borrower)
        print("Days Late:", self.days_late)
    def calculate_late_fee(self):
        if self.days_late <= 5:
            late_fee = self.days_late * 5
        elif 6 <= self.days_late <= 10:
            late_fee = (5 * 5) + (self.days_late - 5) * 10
        else:
            late_fee = (5 * 5) + (5 * 10) + (self.days_late - 10) * 15
        print("Late Fee is:", late_fee)
book1 = LibraryBook(301, "1984", "George Orwell", "Charlie", 8)
book1.display_details()
book1.calculate_late_fee()
book2 = LibraryBook(302, "To Kill a Mockingbird", "Harper Lee", "Diana", 12)
book2.display_details()
book2.calculate_late_fee()
```

OUTPUT:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\college\3-2\AIAC\AIAC LAB\04-02-2026> & "C:/Program Files/Python314/python.exe" "d:/colle
Book ID: 301
Title: 1984
Author: George Orwell
Borrower: Charlie
Days Late: 8
Late Fee is: 55
Book ID: 302
Title: To Kill a Mockingbird
Author: Harper Lee
Borrower: Diana
Days Late: 12
Late Fee is: 105
PS D:\college\3-2\AIAC\AIAC LAB\04-02-2026> █
```

Task 5:

Student Performance Report - Define a function `student\_report(student\_data)` that accepts a dictionary containing student names and their marks. The function should:

- Calculate the average score for each student
- Determine pass/fail status (pass ≥ 40)
- Return a summary report as a list of dictionaries

Use Copilot suggestions as you build the function and format the output

CODE:

```

def student_report(student_data):
    report = []

    for name, marks in student_data.items():
        average = sum(marks) / len(marks)
        status = "Pass" if average >= 40 else "Fail"

        report.append({
            "Student Name": name,
            "Average Score": round(average, 2),
            "Status": status
        })

    return report

students = {
    "Alice": [45, 50, 60],
    "Bob": [30, 35, 40],
    "Charlie": [70, 80, 90]
}

result = student_report(students)
for r in result:
    print(r)

```

OUTPUT:

```

PS D:\college\3-2\AIAC\AIAC LAB\04-02-2026> & "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/04-02-2026/5.1and6.1.py"
{'Student Name': 'Alice', 'Average Score': 51.67, 'Status': 'Pass'}
{'Student Name': 'Bob', 'Average Score': 35.0, 'Status': 'Fail'}
{'Student Name': 'Charlie', 'Average Score': 80.0, 'Status': 'Pass'}
PS D:\college\3-2\AIAC\AIAC LAB\04-02-2026> 

```

Task 6:

Taxi Fare Calculation-Create Python code that defines a class named `TaxiRide` with attributes: `ride\_id`, `driver\_name`, `distance\_km`, and `waiting\_time\_min`. Implement a method `display\_details()` to print ride details, and a method `calculate\_fare()` where:

- ₹15 per km for the first 10 km
- ₹12 per km for the next 20 km
- ₹10 per km above 30 km
- Waiting charge: ₹2 per minute

Create a ride object, display details, and print the total fare.

CODE:

```
class TaxRide:
    def __init__(self, ride_id, driver_name, distance_km, waiting_time_min):
        self.ride_id = ride_id
        self.driver_name = driver_name
        self.distance_km = distance_km
        self.waiting_time_min = waiting_time_min
    def display_details(self):
        print("Ride ID:", self.ride_id)
        print("Driver Name:", self.driver_name)
        print("Distance (km):", self.distance_km)
        print("Waiting Time (min):", self.waiting_time_min)
    def calculate_fare(self):
        if self.distance_km <= 10:
            fare = self.distance_km * 15
        elif 11 <= self.distance_km <= 20:
            fare = (10 * 15) + (self.distance_km - 10) * 12
        else:
            fare = (10 * 15) + (10 * 12) + (self.distance_km - 20) * 30
        fare += self.waiting_time_min * 2
        print("Total Fare is:", fare)
ride1 = TaxRide(401, "Eve", 18, 5)
ride1.display_details()
ride1.calculate_fare()
ride2 = TaxRide(402, "Frank", 25, 10)
ride2.display_details()
ride2.calculate_fare()
```

OUTPUT:

```
PS D:\college\3-2\AIAC\AIAC LAB\04-02-2026> & "C:/Program Files/Py
Ride ID: 401
Driver Name: Eve
Distance (km): 18
Waiting Time (min): 5
Total Fare is: 256
Ride ID: 402
Driver Name: Frank
Distance (km): 25
Waiting Time (min): 10
Total Fare is: 440
```

Task 7:

Statistics Subject Performance - Create a Python function `statistics\_subject(scores\_list)` that accepts a list of 60 student scores and computes key performance statistics. The function should return the following:

- Highest score in the class
- Lowest score in the class
- Class average score
- Number of students passed (score $\geq$ 40)

- Number of students failed (score < 40)

Allow Copilot to assist with aggregations and logic

CODE:

```
178 #7
179 def statistics_subject(scores_list):
180     if len(scores_list) != 60:
181         raise ValueError("scores_list must contain exactly 60 scores.")
182
183     highest_score = max(scores_list)
184     lowest_score = min(scores_list)
185     average_score = sum(scores_list) / len(scores_list)
186     passed_count = sum(1 for score in scores_list if score >= 40)
187     failed_count = len(scores_list) - passed_count
188
189     return {
190         "Highest Score": highest_score,
191         "Lowest Score": lowest_score,
192         "Class Average": round(average_score, 2),
193         "Students Passed": passed_count,
194         "Students Failed": failed_count
195     }
196
197 scores_list = [
198     78, 45, 62, 39, 85, 91, 56, 73, 40, 67,
199     88, 34, 59, 47, 92, 81, 36, 55, 69, 74,
200     53, 61, 95, 28, 49, 77, 64, 83, 31, 46,
201     72, 58, 90, 33, 41, 68, 80, 52, 37, 60,
202     76, 44, 86, 29, 50, 71, 63, 82, 32, 48,
203     70, 57, 93, 35, 42, 65, 79, 54, 30, 38
204 ]
205 result = statistics_subject(scores_list)
206 print(result)
```

OUTPUT:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS D:\college\3-2\AIAC\AIAC LAB\04-02-2026> & "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/04-02-2026/5.1and6.1.py"
{'Highest Score': 95, 'Lowest Score': 28, 'Class Average': 59.88, 'Students Passed': 48, 'Students Failed': 12}
PS D:\college\3-2\AIAC\AIAC LAB\04-02-2026>
```

Task 8:

Use AI to generate two solutions for checking prime numbers:

- Naive approach(basic)
- Optimized approach

Prompt:

“Generate Python code for two prime-checking methods and explain how the optimized version improves performance.”

Expected Output:

- Code for both methods.
- Transparent explanation of time complexity.

- Comparison highlighting efficiency improvements.

Code:

```
#generate python code for two prime-checking methods and explain how optimized version improves performance
def is_prime_basic(n):
    if n <= 1:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
def is_prime_optimized(n):
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True
n = 9
print(f"Is {n} prime (basic method)? {is_prime_basic(n)}")
print(f"Is {n} prime (optimized method)? {is_prime_optimized(n)}")
```

Output:

```
8
PS D:\college\3-2\AIAC\AIAC LAB\04-02-2026> & "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/04-02-2026/5.1and6.1.py"
Is 9 prime (basic method)? False
Is 9 prime (optimized method)? False
PS D:\college\3-2\AIAC\AIAC LAB\04-02-2026> □
```

Task 9:

Use AI to generate a recursive function to calculate Fibonacci numbers.

Instructions:

1. Ask AI to add clear comments explaining recursion.
2. Ask AI to explain base cases and recursive calls.

Expected Output:

- Well-commented recursive code.
- Clear explanation of how recursion works.
- Verification that explanation matches actual execution.

Code:

```
#9
#generate well commented code for the fibanocci series using recursion
def fibonacci(n):
    # Base case: the first two Fibonacci numbers are 0 and 1
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    else:
        # Recursive case: sum of the two preceding numbers
        return fibonacci(n - 1) + fibonacci(n - 2)
# Example usage: print the first 10 Fibonacci numbers
for i in range(10):
    print(fibonacci(i))
```

Output:

```
> & "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/04-02-2026/5.1and6.1.py"
0
1
1
2
3
5
8
13
21
34
```

Task 10:

Use AI to generate a Python program that reads a file and processes data.

Prompt:

“Generate code with proper error handling and clear explanations for each exception.”

Expected Output:

- Code with meaningful exception handling.
- Clear comments explaining each error scenario.
- Validation that explanations align with runtime behavior.

Code:

```
253 #10
254 #Generate code with proper error handling and clear explanations for each exception, Well Commented Code
255 def divide_numbers(a, b):
256     try:
257         # Attempt to perform the division
258         result = a / b
259         return result
260     except ZeroDivisionError:
261         # This exception is raised when trying to divide by zero
262         print("Error: Cannot divide by zero. Please provide a non-zero divisor.")
263     except TypeError:
264         # This exception is raised when the inputs are not numbers
265         print("Error: Invalid input type. Please provide numeric values for both a and b.")
266     except Exception as e:
267         # Catch any other unforeseen exceptions and print the error message
268         print(f"An unexpected error occurred: {e}")
269 # Example usage:
270 print(divide_numbers(10, 2)) # Should print 5.0
271 print(divide_numbers(10, 0)) # Should print an error message about dividing by zero
272 print(divide_numbers(10, 'a')) # Should print an error message about invalid input type
273
274
275
276
```

Output:

```
> & "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/04-02-2026/5.1and6.1.py"
5.0
Error: Cannot divide by zero. Please provide a non-zero divisor.
None
Error: Invalid input type. Please provide numeric values for both a and b.
None
PS D:\college\3-2\AIAC\AIAC LAB\04-02-2026> 
```